



**UNIVERSIDAD TECNOLÓGICA DE
QUERÉTARO**

**Desarrollo para Dispositivos Inteligentes
(SA)**

Profesor: Gerardo Ramírez Villarreal

Alumnos:

Andrea Romero García

Tabla de contenido

REPORTE DE EVALUACIÓN 2 - ECOSISTEMA COMPLETO: TELÉFONO + WEARABLE + SMART TV	3
SECCIÓN 1: DOCUMENTACIÓN DE SEGURIDAD (SA.4).....	3
1.1 Validación y Checklist Técnico	3
1.2 Cumplimiento de la LFPDPPP y Privacidad	3
SECCIÓN 2: PLAN Y REPORTE DE PRUEBAS (SA.5)	4
SECCIÓN 3: REPORTE DE CONFIGURACIÓN DE HERRAMIENTAS Y EMULADORES	7
(SA.6) 3.1 Configuración de herramientas (SA.6.A).....	7
3.2 Configuración de emuladores y resolución de problemas (SA.6.B).....	7
FIRMA DE CONFORMIDAD	8

REPORTE DE EVALUACIÓN 2 - ECOSISTEMA COMPLETO: TELÉFONO + WEARABLE + SMART TV

Asignatura: Desarrollo para Dispositivos Inteligentes

Programa: Ingeniería en Desarrollo y Gestión de Software, Noveno Cuatrimestre

Fecha: 3 de agosto de 2026

SECCIÓN 1: DOCUMENTACIÓN DE SEGURIDAD (SA.4)

1.1 Validación y Checklist Técnico

Validación de event.origin: Dentro del código de la aplicación (PWA), se implementó la validación estricta de la propiedad event.origin al utilizar la API de BroadcastChannel, garantizando que solo se procesen mensajes provenientes de orígenes confiables y previniendo la inyección de datos maliciosos.

Checklist de seguridad PWA: La aplicación cumple con las políticas de Content Security Policy (CSP) mediante las etiquetas meta correspondientes (default-src, connect-src, media-src). Todo el ecosistema opera bajo protocolo HTTPS, hace uso de Subresource Integrity (SRI) y el repositorio no contiene archivos sensibles versionados (.env, .jks, API keys).

1.2 Cumplimiento de la LFPDPPP y Privacidad

Datos personales manejados: La aplicación procesa datos de salud simulados generados por el wearable, específicamente ritmo cardíaco, pasos y calorías quemadas.

Base legal: El tratamiento de la información se sustenta en el consentimiento expreso del usuario final para fines estrictamente de monitoreo personal en tiempo real, en cumplimiento con la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP).

Aviso de privacidad: El responsable del tratamiento de los datos es el alumno desarrollador del presente proyecto. La finalidad única de la recolección es la visualización de las métricas en el ecosistema de dispositivos. El usuario puede ejercer en cualquier momento sus derechos ARCO (Acceso, Rectificación, Cancelación y Oposición) a través de los canales de contacto del desarrollador.

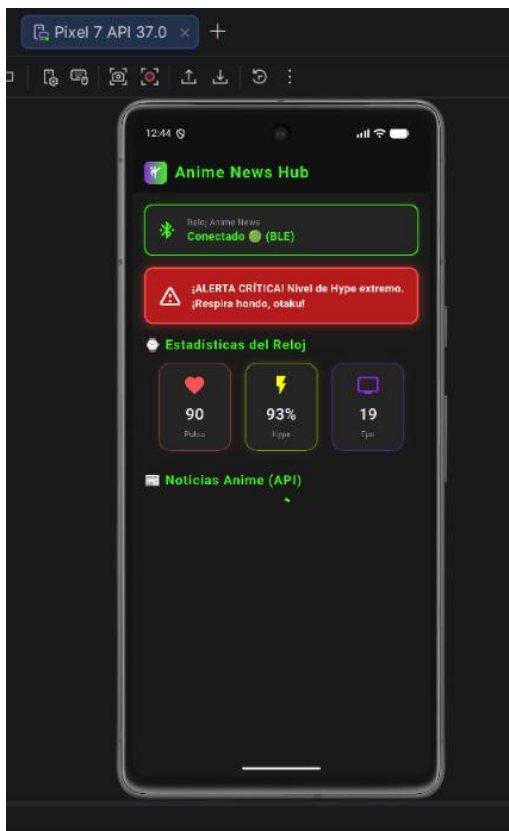
Plan de retención de datos: Los datos de salud se procesan de manera efímera y se almacenan únicamente en la memoria local (caché/estado) del dispositivo durante la ejecución de la app. Al cerrar la aplicación o desconectar el dispositivo BLE, los datos generados en la sesión son eliminados definitivamente, garantizando la privacidad del usuario.

SECCIÓN 2: PLAN Y REPORTE DE PRUEBAS (SA.5)

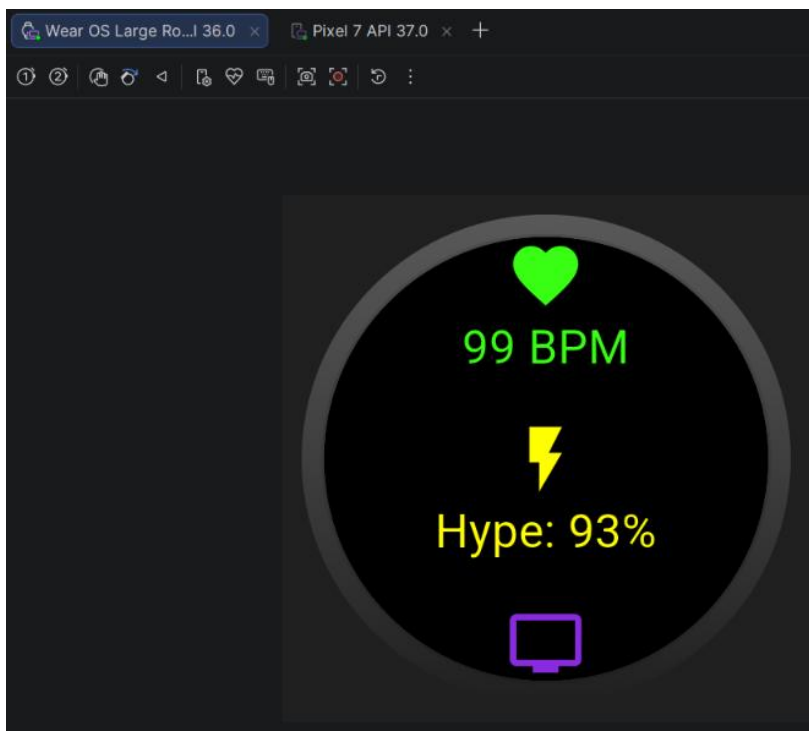
A continuación, se presenta la matriz de los 10 casos de prueba ejecutados sobre el ecosistema. Todos los casos resultaron exitosos.

- Prueba 01 (Prueba API P2.5): La aplicación móvil recupera y muestra datos provenientes de la API del caso de estudio, manejando adecuadamente las excepciones cuando no hay conexión a internet.
- Prueba 02 (Prueba BLE NOTIFY P2.6): Se verificó que los datos por el wearable (Wear OS) son transmitidos y recibidos por el teléfono móvil en tiempo real a través de notificaciones BLE.
- Prueba 03 (Prueba D-pad PWA): La navegación en la Smart TV mediante las flechas del teclado (Arriba, Abajo, Izquierda, Derecha) mueve correctamente el foco visual, y la tecla Enter/OK selecciona la tarjeta activa actualizando el contenido multimedia.
- Prueba 04 (Prueba Modo Offline): Al simular la pérdida de red en Chrome DevTools, el Service Worker interviene y sirve la estructura de la aplicación desde la caché local sin romper la interfaz.
- Prueba 05 (Prueba de Sincronización): Las interacciones realizadas en el teléfono móvil se ven reflejadas en la pantalla de la Smart TV con una latencia menor a los 2 segundos establecidos.
- Prueba 06 (Prueba Límite de Grid): Se comprobó la lógica de navegación; al intentar mover el foco más allá de los bordes del grid (2x2) en la TV, el foco se mantiene estable y la aplicación no presenta errores.
- Prueba 07 (Prueba UI de Conexión BLE): La interfaz del teléfono actualiza y muestra claramente los estados de conexión del Bluetooth (buscando, conectado, error, desconectado).
- Prueba 08 (Prueba de Desconexión): Al forzar la desconexión o apagar el wearable, la aplicación del teléfono maneja la excepción, no sufre un cierre inesperado (crash) y notifica al usuario en pantalla.
- Prueba 09 (Prueba de Control de Sensores): El botón de Iniciar/Detener ubicado en la interfaz del wearable Wear OS controla de forma precisa el inicio y la interrupción del flujo de datos enviados.
- Prueba 10 (Prueba Ecosistema 3 Dispositivos): El teléfono, el wearable y la Smart TV operaron simultáneamente y sin interrupciones o caídas de rendimiento durante una ventana de prueba continua de 5 minutos.

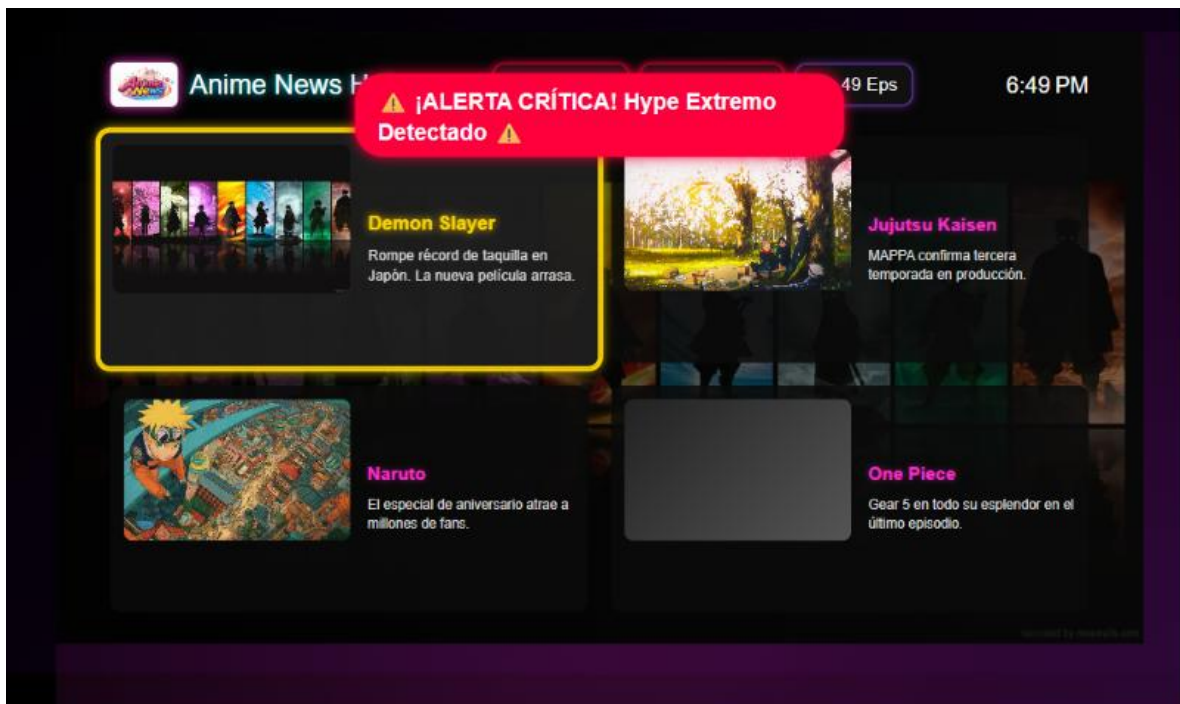
[1: Teléfono corriendo app]



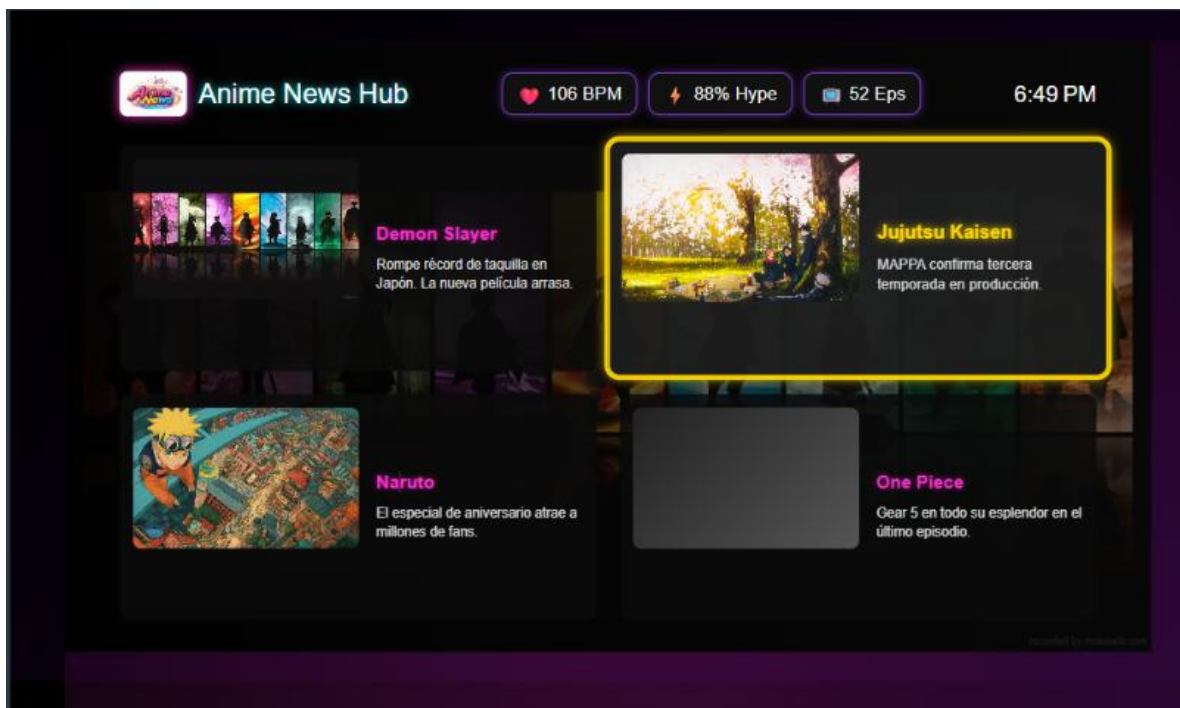
[2: Wear OS corriendo app]



[3: PWA en modo TV]



[4: Interfaz de conexión BLE exitosa]



SECCIÓN 3: REPORTE DE CONFIGURACIÓN DE HERRAMIENTAS Y EMULADORES

(SA.6) 3.1 Configuración de herramientas (SA.6.A)

Versiones SDK: Se utilizó Flutter SDK estable con su respectiva versión de Dart SDK para compilar los proyectos móviles.

Entorno de desarrollo: Android Studio con los plugins oficiales de Flutter y Dart actualizados. Para la PWA se utilizó Visual Studio Code con extensiones de formateo y validación web. Adicionalmente, se integró ffmpeg para la optimización de los recursos multimedia del caso de estudio.

Dependencias principales (pubspec.yaml): El proyecto integra flutter_blue_plus (v1.33.0) para el manejo de Bluetooth de baja energía, http para el consumo de la API REST y provider para la gestión de estados.

Pasos de instalación reproducibles: Para desplegar el entorno, es necesario clonar el repositorio unificado, ejecutar flutter clean y flutter pub get en los directorios correspondientes (app y wearable), iniciar los emuladores detallados a continuación, y levantar la PWA utilizando un servidor de desarrollo local o Live Server.

3.2 Configuración de emuladores y resolución de problemas (SA.6.B)

Emulador de Teléfono: Dispositivo virtual configurado en Android Studio, compilando contra API Level 34, con arquitectura x86_64.

Emulador Wear OS: Dispositivo virtual tipo Round (Circular), compilando contra API Level 30, con asignación de 1GB de RAM.

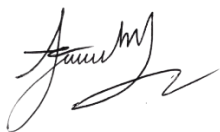
Emulador Smart TV: Configurado directamente a través del Device Mode en Chrome DevTools, estableciendo una resolución fija de 1920x1080 (10-foot UI) y ocultando las barras de desplazamiento. Troubleshooting Documentado (Problemas y Soluciones):

Durante la integración inicial del ecosistema, al intentar ejecutar la aplicación móvil (flutter run), la consola de Gradle arrojó un error en la tarea "checkDebugAarMetadata" relacionado con el paquete flutter_blue_plus. El problema indicaba que las bibliotecas de AndroidX requeridas por el paquete (como androidx.fragment y androidx.window) exigían compilar contra la versión 34 de la API de Android, mientras que el proyecto estaba configurado en la versión 33. Solución aplicada: Me dirigí al archivo android/app/build.gradle de mi proyecto y actualicé los valores de compileSdkVersion y targetSdkVersion a 34 dentro de los bloques correspondientes. Tras ejecutar un "flutter clean" y volver a compilar, el proyecto levantó exitosamente en el emulador, permitiendo la comunicación GATT sin problemas.

FIRMA DE CONFORMIDAD

Por medio del presente, confirmo que el ecosistema y la documentación adjunta cumplen con los lineamientos de la Evaluación 2.

Firma del Alumno

A handwritten signature in black ink, appearing to be 'Juan M.' followed by a stylized flourish.

Fecha: 3 de agosto de 2026